



Lab #10: Interrupts of 8086

Question 1:

Consider the following program.

```
.data
a BYTE -5
b BYTE 15
c WORD 002Ah
d DWORD 0000FFFFh

.code
main PROC
    mov al, a
L1: add al, b
    movzx eax, c
L2: dec eax
    mov eax, d
L3: ror eax, 4
L4: sub ax, c
    exit
main ENDP
END main
```

Show the hexadecimal value of EAX and the status of the ZF, SF, CF, and OF at the following:

1. After executing of the instruction located at label L1

EAX (HEX)	ZF	SF	OF	CF
0000 000A	0	0	0	1

2. After executing of the instruction located at label L2

EAX (HEX)	ZF	SF	OF	CF
0000 0029	0	0	0	1

3. After executing of the instruction located at label L3

EAX (HEX)	ZF	SF	OF	CF
F000 0FFF	0	0	0	1

4. After executing of the instruction located at label L4

EAX (HEX)	ZF	SF	OF	CF
F000 0FD5	0	0	0	0

Question 2:

Consider the following program.

```
.data
    nums BYTE 12h, 34h, 56h, 78h, 9Ah
    sum DWORD 0

.code
main PROC
    mov esi, OFFSET nums
    xor eax, eax
    mov ecx, 5

Label1:
    CALL AddToSum
    inc esi
    loop Label1

    mov sum, eax

Label2:
    mov dx, 0AF0Ah
    xor ax, dx

Label3:
    mov bx, sum
    and bx, 0xFF00

    exit
main ENDP

AddToSum PROC
    add al, [esi]
    ret
AddToSum ENDP

END main
```

1. When the control reaches `Label1` (just before executing `loop Label1` for the last iteration), what will be the values of the following registers:

Register, Memory location, or flag bit	Value of the register or memory location or flag (in HEX)
EAX	000000AE
EBX	00000000
ECX	00000001
ESI	offset esi

2. When the control reaches `Label2` (just after `xor ax, dx`), what will be the values of the following registers:

Register, Memory location, or flag bit	Value of the register or memory location or flag (in HEX)
AX	AFA4
DX	AF0A
SF	1

3. When the control reaches Label3 (just before executing `bx, 0xFF00`), what will be the values of the following registers and flags:

Register, Memory location, or flag bit	Value of the register or memory location or flag (in HEX)
AX	AFA4
BX	00AE
CF	0

Question 3:

Write an assembly program that processes an array of 10 integers. The program should include three procedures:

1. InputArray:

This procedure takes an offset of an array as input in ESI and prompts the user to input 10 positive two-digit integers (10–99). Assume that the user only inputs valid numbers. The numbers are stored in an array called Values.

2. TransformIfSameDigit:

This procedure takes a two-digit number as input and checks if the digits are the same (e.g., 11, 22). If the digits are the same, it converts the number to its negative equivalent and returns it in EAX. If not, it returns the original number unchanged in EAX.

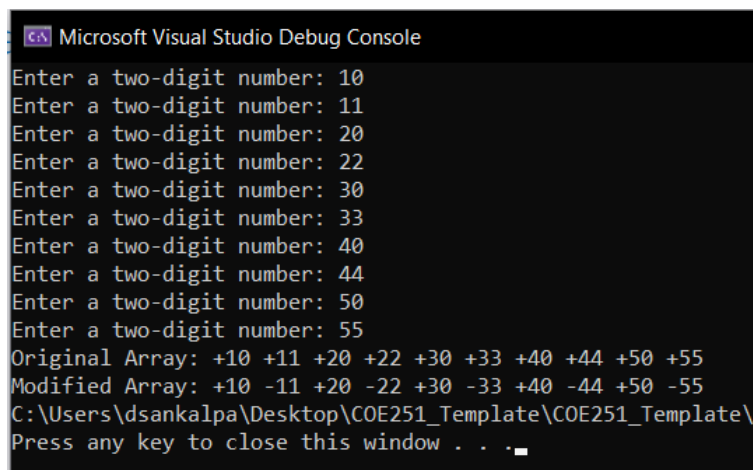
3. PrintArray:

This procedure takes an offset of an array as input through ESI and prints all its elements.

The main program should:

- Prompt the user to enter 10 positive two-digit integers to fill an array.
- Iterate through the array, calling `TransformIfSameDigit` for each element.
- Replace numbers with identical digits in the array with their negative equivalents.
- Print both the original and modified arrays using the `PrintArray` procedure.

Sample screenshot:



```
Microsoft Visual Studio Debug Console
Enter a two-digit number: 10
Enter a two-digit number: 11
Enter a two-digit number: 20
Enter a two-digit number: 22
Enter a two-digit number: 30
Enter a two-digit number: 33
Enter a two-digit number: 40
Enter a two-digit number: 44
Enter a two-digit number: 50
Enter a two-digit number: 55
Original Array: +10 +11 +20 +22 +30 +33 +40 +44 +50 +55
Modified Array: +10 -11 +20 -22 +30 -33 +40 -44 +50 -55
C:\Users\dankalpa\Desktop\COE251_Template\COE251_Template\
Press any key to close this window . . .
```

Code:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

str1 byte "Enter a two-digit number: ", 0
str2 byte "Original Array: ", 0
str3 byte "Modified Array: ", 0
str4 byte " ", 0
Values dword 10 DUP(?)
divisor dword 10

.code

main PROC

mov edx, offset str1

call InputArray

mov edx, offset str2
call writestring

call PrintArray
call crlf

mov edx, offset str3
call writestring

mov esi, offset Values
mov ecx, lengthof Values

again2:

call TransformIfSameDigit
mov [esi], ebx
add esi, 4
loop again2

call PrintArray

exit

main ENDP

InputArray proc

mov esi, offset Values
mov ecx, lengthof Values

again1:

call writestring
call readint
mov [esi], eax
add esi, 4
loop again1

```

        ret
InputArray endp

TransformIfSameDigit proc
    mov ebx, [esi]

    mov eax, ebx
    mov edx, 0
    div divisor
    cmp edx, eax
    jne repeatLoop

    neg ebx

repeatLoop:
    ret
TransformIfSameDigit endp

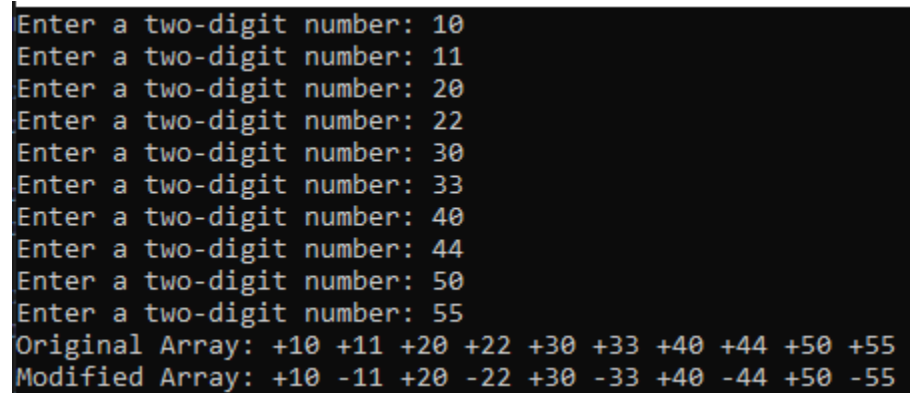
PrintArray proc
    mov edx, offset str4

    mov esi, offset Values
    mov ecx, lengthof Values

again3:
    mov eax, [esi]
    call writeint
    call writestring
    add esi, 4
    loop again3
    ret
PrintArray endp
END main

```

Screenshot:



```

Enter a two-digit number: 10
Enter a two-digit number: 11
Enter a two-digit number: 20
Enter a two-digit number: 22
Enter a two-digit number: 30
Enter a two-digit number: 33
Enter a two-digit number: 40
Enter a two-digit number: 44
Enter a two-digit number: 50
Enter a two-digit number: 55
Original Array: +10 +11 +20 +22 +30 +33 +40 +44 +50 +55
Modified Array: +10 -11 +20 -22 +30 -33 +40 -44 +50 -55

```